

real-sw-p1-full

An AgentMPI run analysis

Abstract

This is the serial baseline of the collaborative software-development experiment: one real agent, given the whole `minidb` specification, writing all eight modules itself. It passed 59 of 60 acceptance cases at the first attempt and 60 of 60 after one repair round, in 1,213.0s for \$0.4379 — against 499.4s and \$1.1445 for the eight-rank baseline `real-sw-p8-full`, which also reached 60/60. Eight agents therefore bought 2.43× the speed for 2.61× the money and 31.1× the prompt tokens. The run’s real value is as a measurement of what AgentMPI costs when it is doing nothing: of the 1,212.9986s of wall clock, 1,210.228s is time a worker held a claim and 1,212.896s is inside the two `agent.call` latencies, leaving **2.768s** — **0.228%** — for broker dispatch, contract validation, result handling, two invocations of the acceptance oracle, and every fabric write the protocol performed. The protocol proper is thirteen event rows and one degenerate broadcast. Two things qualify that figure and both matter: the $p = 1$ path in the harness branches away before the RMA window is ever created, so this is the floor of the runtime rather than the fixed cost of the full pipeline; and 47 of the run’s 60 events are not this job’s at all, but a shared worker pool registering ranks 0–7 into a world of size 1 on a 244.7s cycle, which the viewer then reports as seven ranks suspected of `fail_stop` in a run with zero failures. The structural result the run is uniquely placed to give is about verification, not overhead: every one of the 60 acceptance cases enters through `minidb.query`, so no case can run until all eight modules exist. At $p = 1$ that gate opens after every agent call; at $p = 8$ it opens once per round, at rank 0, after the integration barrier, and never for an individual rank at all.

1 What this run is

property	value
run	<code>real-sw-p1-full</code>
experiment	<code>software</code>
campaign label	<code>real-sw</code>
declared world size	1
ranks appearing in the log	8
executors	<code>broker</code> × 1, <code>worker</code> × 47
events	60
wall time	1,213.00s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	1.00×	total busy time over wall time
parallel efficiency	99.8%	achieved parallelism per rank
idle fraction	0.2%	rank-seconds paid for and unused
peak ranks busy	1	of 1 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	1.00×	slowest rank’s busy time over the mean
coordination share	0.0%	rank-seconds blocked in collectives, of those available
coordination in flight	0.0%	wall clock with any rank inside a collective
rank reattachments	40	fresh agent processes that took over a rank role
agent calls	2	invocations that reached a model
agent latency p50 / p95	606.45 / 644.94 s	per invocation
messages	0	0 eager, 0 rendezvous
tokens sent	0	0 deferred under rendezvous
tokens in / out	3,443 / 28,502	prompt and completion
cost	\$0.4379	0.0154 \$/kTok out

3 What the analysis notices

- **[warning]** Ranks 1, 2, 3, 4, 5, 6, 7 appear in the log without participating; the declared world size is 1. Shared worker pools let a worker register against the wrong job, which inflates the apparent rank count.
- **[warning]** 1 collective(s) completed but recorded no duration (**bcast**), so they contribute nothing to the coordination figures even though every participant blocked in them. The reported coordination share is short by exactly their blocking; it can be recovered from the event timestamps.
- **[note]** Too few distinct output sizes to fit β , so the reported latency parameters are medians rather than a regression.
- **[ok]** 40 rank reattachment(s) occurred, up to incarnation 9: agent processes were replaced mid-run while the rank roles, their mailboxes, and their context accounts persisted.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	1,210.23	99.8	2	2	0	0	0	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

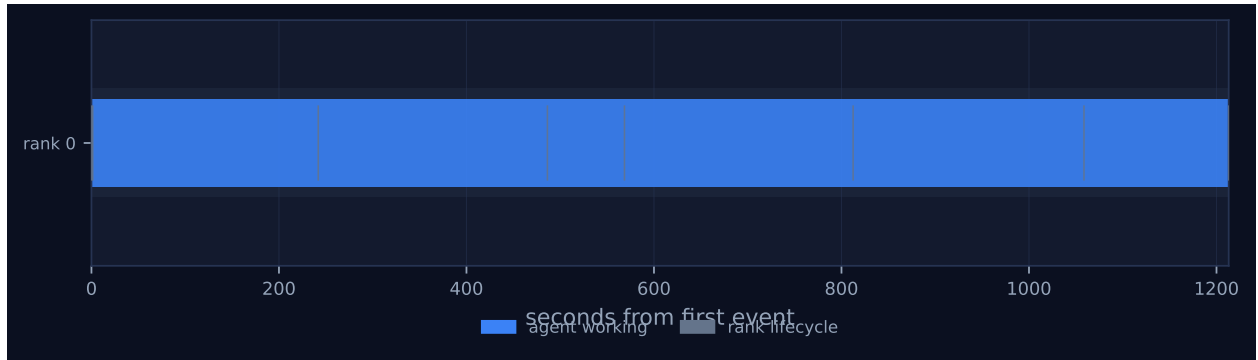


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

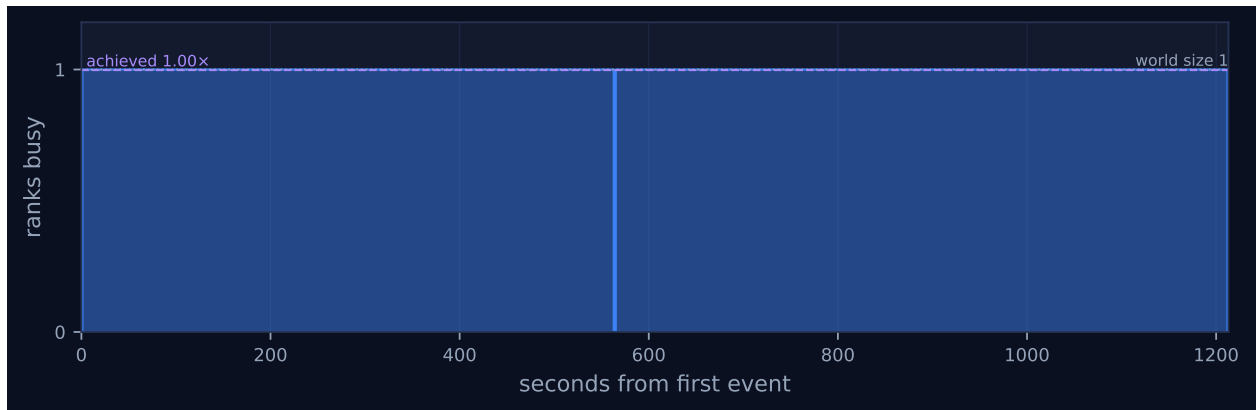


Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

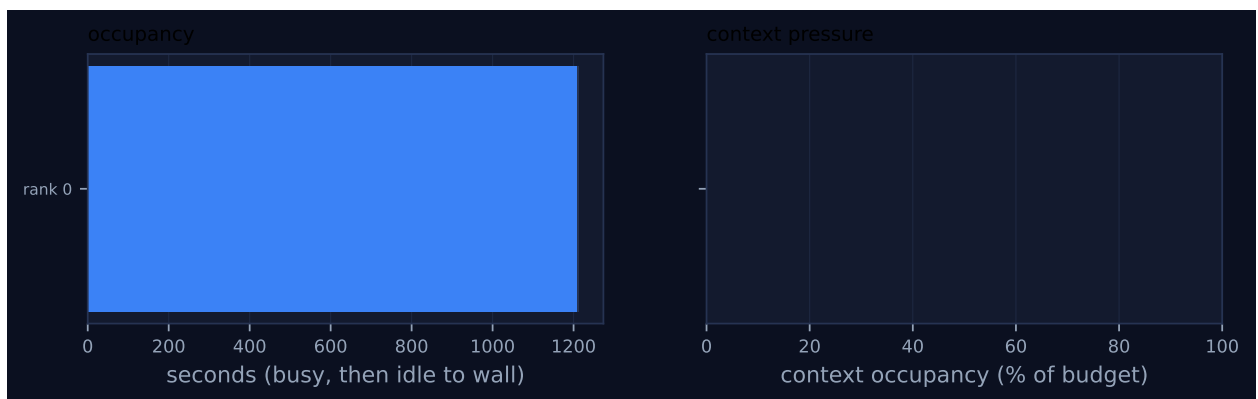


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

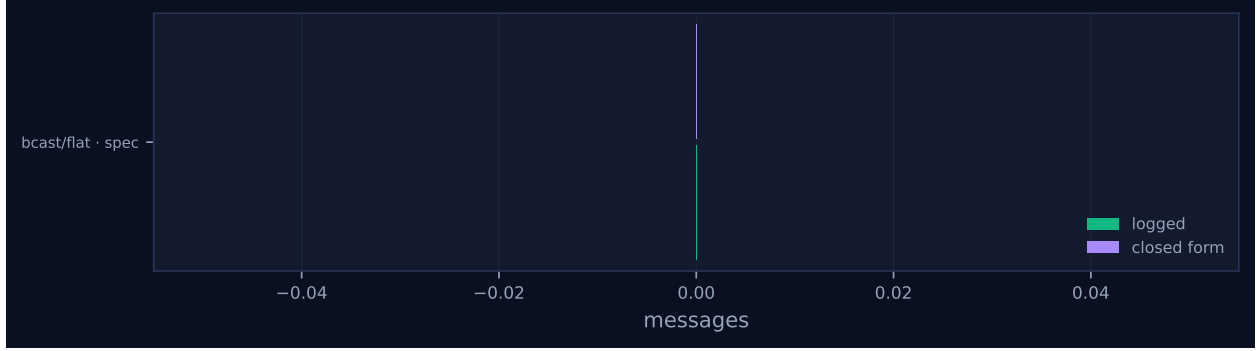


Figure 4: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
bcast	flat	spec	1	1	0	0	0	0	0	0.00	0.00	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is the simplest picture in the archive and almost all of it is somebody else’s run. Rank 0’s lane holds two blue work bars that between them cover 1,210.23s of the 1,213.00s span, and the seam between them — at $t = 563.7$ s — is so narrow that the viewer draws the pair as one continuous bar. The other seven lanes contain no work at all: only evenly spaced grey lifecycle ticks, five to nine per lane, marching across the full width of the plot. Those lanes should not exist. The declared world size is 1, the job created exactly one rank, and ranks 1 through 7 have `busy_s` of 0.0 and `n_agent_calls` of 0. They are the subject of the run’s one [\[warning\]](#) finding and of Section 7.

Figure 2 is the corresponding flat line: one rank busy for 99.77% of the run, zero for the remaining 0.23%, with a single faint vertical mark at $t \approx 0$ for the only collective the run performed. Figure 3 shows rank 0 at 99.8% occupancy and a right-hand context panel that is empty at 0%; the latter is not a finding about context pressure but a consequence of the harness never admitting anything into a context account, exactly as in the $p = 8$ runs. Figure 4 has one point on it, and it sits on the line: a flat broadcast at $p = 1$ is predicted to send zero messages and sent zero.

The interesting reading is of the seam rather than the bars. Laying the thirteen events that belong to this job end to end, with time relative to `job.create`:

t (s)	event	what it covers
0.0000	<code>job.create</code>	fabric schema, blob store, spool
0.0012	<code>rank.init</code>	rank 0 registers, executor <code>broker</code>
0.0013	<code>coll.bcast</code>	the specification, $p = 1$: 0 rounds, 0 messages
0.0055	<code>broker.enqueue</code>	prompt <code>single:r1</code> , 1,706 tokens
1.3962	<code>broker.claim</code>	a pool worker takes the task
563.5329	<code>broker.complete</code>	14,066 result tokens
563.6854	<code>agent.call</code>	contract <code>Bundle</code> validated, latency 563.682s
563.7367	<code>broker.enqueue</code>	prompt <code>single:r2</code> , 1,737 tokens
564.3652	<code>broker.claim</code>	
1212.4560	<code>broker.complete</code>	14,436 result tokens
1212.9493	<code>agent.call</code>	latency 649.214s
1212.9983	<code>rank.finalize</code>	
1212.9986	<code>job.finish</code>	

Everything AgentMPI did in this run is in that table. The two gaps that are not agent time are 0.0055s before the first prompt and 0.0513s and 0.0490s after each completion; the latter two are where the harness wrote eight files, copied the oracle in, ran it out of process, and built the next prompt. Running the archived round-1 tree against `experiments/minidb/acceptance.py` in a scratch directory takes 0.040–0.050s per invocation over three trials, which accounts for essentially the whole of each gap. That is the measurement behind the abstract’s claim: at $p = 1$ the fabric’s own contribution to wall clock is not merely small, it is smaller than the cost of starting a Python interpreter.

7 Interpretation

What is true by construction

The run has one rank because `-ranks 1` was passed, and at $p = 1$ `build_harness` broadcasts the specification and then returns into `_single_rank`, which is a different code path from the one every other software run takes. That branch happens before `mpi.win_create`, so this run performs no window operation of any kind: the `kind_counts` contain no `win.*` event, no `coll.barrier`, no `coll.gather`, no `coll.scatter` and no `ft.agree`. The `shared_interfaces`, `locks` and `review` flags are all true in the config and all inert. `agreed_green` is `null` for both rounds for the same reason: `agree` is never called, so the loop terminates on the oracle’s verdict directly rather than on a collective decision. The two rounds are the configured maximum, and the run used both.

Three headline numbers are therefore trivially true and should not be read as results. The imbalance of $1.00\times$ and the coordination share of 0.0% are what a one-rank world must report. The achieved parallelism of $1.00\times$ is a statement that one rank was busy almost all the time, not that anything ran in parallel.

The fixed overhead, stated three ways

The question this run is here to answer is what the protocol costs when there is nothing for it to coordinate. There are three defensible boundaries and they give three numbers.

The narrowest is wall clock outside the agent calls: $1,212.9986 - 1,212.8960 = \mathbf{0.1026\text{ s}}$, or 0.00846 % of the run. Most of that is the oracle, which is the experiment rather than the protocol.

The widest useful boundary is wall clock outside the interval a worker actually held the task. Summing `broker.claim` to `broker.complete` over the two calls gives 1,210.228 s, which matches the independently computed `busy_s` of 1,210.227 s for rank 0 to a millisecond. The complement is **2.768 s**, **0.228 %** of the run, and it decomposes cleanly: 2.019 s of dispatch wait, from `broker.enqueue` to `broker.claim` (1.391 s and 0.629 s); 0.646 s of result handling, from `broker.complete` to the `agent.call` event that follows it (0.153 s and 0.493 s), which is where the JSON is decoded and the `Bundle` contract is checked; and the 0.103 s above. Zero contract violations and zero retries were recorded, so the validation cost is the cost of a passing check.

The third boundary is the one the paper cares about and it is a count rather than a duration: thirteen event rows and a 159,744-byte SQLite fabric for a twenty-minute run. The only collective is a broadcast to nobody, correctly logged as `rounds: 0, messages_sent: 0, wall_s: 0.0`.

The honest caveat is the one already stated: because `_single_rank` bypasses the RMA window, this is the floor of the runtime and not the fixed cost of the pipeline the $p = 8$ runs execute. A $p = 1$ run of the *full* pipeline would additionally create a window, publish eight interfaces into it, fence against itself, and read fourteen dependency slots that it had itself written; the archive contains no such run. What can be said from the $p = 8$ traces is that those operations are individually sub-millisecond — in `sw-rev-smoke`, 100 RMA events occur inside a 0.324 s run — so the extrapolated floor would still be well under a second. The two-decimal-order claim survives; the exact figure is for the branch this run took.

Continuous verification is available at $p = 1$ and unavailable at $p = 8$

This is the asymmetry the project asserts and the only run in the archive that isolates it. The assertion is usually made loosely — a single agent can test as it goes — and the harness lets it be made exactly.

The oracle's entry point is `minidb.query`, and every one of the 60 cases in `CASES` calls it. `query` lives in `api.py`, which the module table declares dependent on `parser`, `planner`, `engine` and `errors`; those in turn require `tokens`, `nodes` and `functions`. Six of the eight modules have declared dependencies, and `run_all` returns `importable: false` with zero cases attempted if `import minidb` raises. The consequence is not that testing a single module in isolation is inconvenient at $p = 8$; it is that the acceptance suite contains **no case that any individual rank could run**, at any point before the integration barrier, no matter how complete its own module was. The three smoke runs in this family demonstrate the same thing from the other side: their surrogate modules are syntactically valid stubs and the suite still reports zero of sixty, because `minidb.api` exposes no `query`.

At $p = 1$ that gate is open after every agent call, and the trace shows the loop closing in 0.05 s. Round 1 scored 59/60; the single failure, `empty_table_select`, was attributed to `planner`; round 2 scored 60/60. At $p = 8$ the same feedback arrives through `barrier` → `gather` → oracle at rank 0 → `bcast` → `scatter`, once per round, and the latency of that loop is set by the slowest rank: in `real-sw-p8-full`, rank 0 finished its module at $t = 85.0$ s and did not learn anything about it until the barrier released at $t = 300.6$ s, a 215.56 s wait. The gap between 0.05 s and 215.56 s is not a protocol inefficiency to be optimised away. It is what "the module's tests cannot run until its dependencies exist" costs, and AgentMPI's contribution is to make the barrier the place where that cost is paid visibly rather than the place where a build silently diverges.

The repair itself shows the second half of the asymmetry, which is granularity. One failing case out of sixty produced a rewrite of all eight files: `planner` grew from 341 to 530 lines, `parser` from 365 to 391, `tokens` from 95 to 148, while `api` fell from 47 to 26 and `errors` from 17 to 8; not one of the eight round-1 digests survives into round 2. A single agent holding the whole system revises the whole system. At $p = 8$ the harness scatters blame per module precisely so that it need not, and only the blamed owners rewrite — although `real-sw-p8-full` cannot be used to demonstrate that, because its oracle report failed to parse and every rank was handed the same synthetic `IMPORT` failure.

The code one agent writes is the most internally consistent in the experiment

The offline re-scoring stored with the result computes two structural measures over the delivered tree, and on both of them this run is the extreme of the five real-agent software runs.

run	lines	defensive constructs	per kloc	mixed-convention files
<code>real-sw-p1-full</code> (r2)	1,752	1	0.57	0
<code>vague-sw-p8-vague-shared</code>	2,203	2	0.91	3
<code>real-sw-p8-full</code>	1,954	7	3.58	1
<code>vague-sw-p8-vague-noshared</code>	3,564	15	4.21	5
<code>real-sw-p8-noshared</code>	2,693	33	12.25	2

Defensive constructs are `getattr` with a default, `hasattr`, and `except` clauses that adapt to a peer’s shape — the syntax of an author who is not sure what the other side exposes. The $p = 1$ tree contains exactly one, in `engine.py`, against 33 in the windowless eight-rank run. The naming audit is starker: zero files in which the same concept is spelled two ways, against two to five everywhere else. This is the mechanical version of "one agent writes both sides of every call", and it is worth being precise about what it does and does not show. It does not show that $p = 1$ produces better code: the acceptance outcome is 60/60 in three of the five runs above, and the oracle cannot tell them apart. It shows that the artefacts of *coordination failure* — runtime introspection of a collaborator, two names for one idea — are absent when there is no collaborator, which locates them as coordination costs rather than as properties of the model or the task. Note also that the $p = 1$ run’s own round 1 tree scored 2.62 defensive constructs per kloc and mixed two conventions; the repair round removed both. Even a single author accumulates inconsistency across a 1,500-line generation and cleans it up on a second pass.

What the run contributes to the sweep

run	p	wall (s)	calls	tokens in	tokens out	USD
<code>real-sw-p1-full</code>	1	1,213.0	2	3,443	28,502	0.4379
<code>real-sw-p8-full</code>	8	499.4	32	107,118	54,871	1.1445
<code>real-sw-p8-noshared</code>	8	528.6	32	94,239	72,935	1.3768
<code>vague-sw-p8-vague-shared</code>	8	401.9	16	54,169	32,304	0.6471
<code>vague-sw-p8-vague-noshared</code>	8	847.5	32	102,613	88,989	1.6427

Every one of these reached 60/60 on re-scoring except `real-sw-p8-noshared` at 59/60. The baseline is therefore not beaten on quality by anything; it is beaten on latency by three of the four and on

price by none of them. The cheapest eight-rank cell, `vague-sw-p8-vague-shared`, is $3.0\times$ faster and $1.48\times$ dearer, which is the most favourable trade in the archive and is favourable partly because it happened to go green in one round.

The prompt-token column is where the parallel decomposition is most expensive, and the ratio is not $8\times$ but $31.1\times$. The specification is 1,526 tokens and it is embedded in every prompt: two prompts at $p = 1$, thirty-two at $p = 8$, each additionally carrying published dependency interfaces, scattered failures and returned reviews. Completion tokens scale far more gently — 28,502 against 54,871, a factor of 1.93 for a tree only 1.12 times longer — because the system being written is the same system either way. Parallelising this task costs prompt tokens roughly in proportion to p and completion tokens hardly at all, and since prompt tokens are the cheap ones the bill only doubles. That decomposition of the cost is the baseline’s main quantitative contribution.

The flagged findings

Of the four findings above, the two `[ok]` marks are correct but one is misleading, the `[note]` is right and newly added, and the `[warning]` is a real defect. All one checkable collective agreed with its closed form, which at $p = 1$ is a statement that zero equals zero. The 40 rank reattachments up to incarnation 9 are described as normal broker behaviour; six of them are rank 0 and 34 are ranks that do not exist in this job, so the number is not a property of this run at all. The note that there are too few distinct output sizes to fit β is exactly the calibration problem discussed above, and it was added to the tooling after this analysis first identified it. The warning about stray ranks is the subject of the next section.

Forty-seven registrations that belong to another job

Forty-seven of the run’s 60 events are `rank.init` with `executor: worker`, and they arrive on a metronome: taking every inter-registration interval longer than 50 s across all eight rank slots gives 35 samples with a median of 244.7 s, a minimum of 82.2 s and a maximum of 247.1 s. A one-rank job that makes two agent calls does not re-register a rank every four minutes. The pool did.

The pool was busy elsewhere. `real-tr-p16-full` was live from 06:47:39 to 08:47:39 and therefore spanned this entire run; inside this run’s window it logged six `broker.expire` events with `waited_s: 5400.0` and six matching `harness.degraded ERR_TIMEOUT` records for `terms:u9` through `terms:u15`. A sixteen-rank job was starving for workers at the same moment that a one-rank job was accumulating eight rank rows. I cannot show from the trace that these are the same workers — registration in one fabric leaves no trace in another, and none of the 48 `rank.init` timestamps here falls within a second of a `rank.init` there — but the coincidence is worth recording for anyone reading either trace.

The defect itself is understood and half-fixed. `RankRuntime.register` accepted any rank index unconditionally; commit 6728d91 now refuses an index outside the world, and its docstring lists the three translation runs where the leak was first seen (worlds of 2, 4 and 8). This run is a fourth instance and the most extreme: seven of the eight rank rows in a world of size 1 are foreign, and ranks 1–7 carry lifetimes of 968–1,209 s in `metrics.json` despite never doing anything.

What this run adds is evidence for the case that commit says it cannot close. Registration is an upsert keyed on rank index, so a worker landing on an index *inside* the world would silently take over the incumbent’s lease and mailbox. Rank 0 received six worker registrations beyond its own broker registration. Two of them are legitimate and identifiable: the one at $t = 1.396$ s immediately

precedes the first `broker.claim`, and the one at $t = 568.53$ s follows the second claim by 4.2 s. The other four — at $t = 241.90, 486.33, 812.41$ and $1,058.66$ s — sit on the same 244.7 s cadence as the seven out-of-world lanes and arrived while a claim was outstanding. If they are pool strays, then the dangerous case did happen here, at least four times, and did no damage only because nothing was waiting in rank 0's mailbox. If they are the claiming worker renewing, they are benign. *The trace cannot distinguish these*, which is exactly the property that makes the case unclosable: a worker holding the right index is indistinguishable from the rank it claims to be. Anyone tempted to consider the registration issue closed by 6728d91 should read these six events first.

The consequence for the viewer is worth seeing rather than describing. The dashboard's stats row reports FAILURES 0. Its rank table reports rank 0 as `finalized` and every other rank as `running, alive: no`, with `fail_stop` in the SUSPECTED column. Seven ranks are flagged as suspected failures in a run that had none, because the failure detector is doing its job on rows that should never have been created.

Two numbers in the artefacts that do not mean what they say

The headline table and the dashboard both report agent latency as $p50 / p95 = 649.21 / 563.68$ s, a p95 below the p50 and a p50 equal to the maximum. With two samples, `RunSummary.as_dict` computes the median as `lat[len(lat) // 2]`, which is the upper of the two, and the p95 as `lat[int(0.95 * (len(lat) - 1))]`, which floors to index 0 and returns the smaller. Both indices are wrong in the same direction for small samples, and for $n = 2$ the reported p95 is the minimum. The same pattern appears in `cost.calibrate`, where `alpha_p99` is 563.682 against an `alpha_p50` of 606.448. This is cosmetic here — the two latencies are printed in full in the log — but it is on the dashboard, and on any run with fewer than about twenty calls the percentile pair should be ignored.

The second is the calibration block, which reports `tokens_per_s: 45.0` and an α/β crossover at 27,290 tokens. Neither is fitted from this run. `calibrate` requires at least three samples with distinct token counts before it runs its least squares; with $n = 2$ it takes the `elif` branch, sets α to the median latency and leaves β at the `CostParams` class default of 1/45 s per token. The run's own throughput is $14,066/563.682 = 24.95$ and $14,436/649.214 = 22.24$ tokens per second, an aggregate of 23.50 — about half the figure the archived calibration reports.

Half of that is now signposted. The calibration block records `n_samples: 2` and, since the regeneration that followed this analysis, `fit_method: "median_only"`, and the findings list above carries a matching [\[note\]](#). What is still not signposted is β itself: `tokens_per_s: 45.0` is printed in the same block as the medians with nothing to distinguish a class constant from a fitted value, and on this run it overstates the observed rate by roughly a factor of two. Any comparison of `tokens_per_s` across runs should first check `fit_method`.

8 Threats to this reading

One shared output digest, and the pool was shared. `real-sw-p8-full` finished at 07:55:25 and this run started at 08:04:14 against the same worker pool, so the contract's contamination test applies. Comparing content hashes across all four archived trees, exactly one agent-authored file coincides: `errors.py` in this run's round 1 is byte-identical to `errors.py` in both rounds of `real-sw-p8-full` (17 lines, a22b8a49e1c7). The harness-written `__init__.py` matches everywhere and is not evidence. Nothing else matches: `engine.py` is 340 lines here against 429 there, `planner.py`

341 against 505, `tokens.py` 95 against 170.

The evidence favours convergence, for three reasons, and I cannot exclude reproduction. First, the shape is wrong for reuse: a worker replaying its previous output would not replay one file of eight, and at $p = 1$ all eight came back in a single JSON object from a single call. Second, `errors.py` is the most constrained artefact in the system — the specification’s module table pins it to “`QueryError(Exception)` and nothing else”, it has no logic, and there is essentially one way to write it. Third, and most usefully, the identical text did not survive: round 2 replaced it with a different 8-line file, so the tree that scores 60/60 contains no byte in common with the $p = 8$ run. The uncomfortable part is that the matching 17 lines are mostly a docstring whose error taxonomy — “malformed SQL, unknown tables or columns, ambiguous column references, mixed-type comparisons, negative LIMIT/OFFSET values and type errors” — is assembled from four separate places in the specification and matches including its line wrapping. That is a stronger coincidence than a bare class statement would be, and the trace records no worker identity, so it cannot be resolved here.

$n = 2$, and the two calls disagree about the model. Every timing statement about the agent rests on two samples whose latencies differ by 15 % and whose throughputs differ by 12 %. The claim that the baseline took 1,213s is exact; the claim that a single agent writing `minidb` takes about twenty minutes is a sample of one, drawn on a pool that was concurrently oversubscribed by a sixteen-rank job, which is a plausible source of the dispatch waits and possibly of latency inside them.

The overhead figure is a floor, not the pipeline’s fixed cost. Restating the caveat because it is the most likely thing to be misquoted: 0.228 % is the overhead of the code path `-ranks 1` actually executes, which contains no window, no barrier, no gather, no scatter and no agreement. It bounds what AgentMPI costs when idle. It does not bound what the eight-rank pipeline’s coordination machinery would cost if run against a single rank, and no run in the archive measures that.

The oracle changed after this run. The suite re-scored offline carries `suite_sha 0eaa6b695b83` and 60 cases; `sw-smoke`, recorded at 07:02, ran a 59-case version. This run’s in-flight and offline scores agree at 60/60, so nothing here depends on the difference, but the 59/60 of round 1 and the 60/60 of round 2 are scores against a suite that was itself under revision during the campaign, and one case in it was later found to contradict the specification.

The provenance commit is not the code that ran. The result names `a803ffd`, which was committed at 08:10:32, six minutes after the run began at 08:04:14. `provenance()` records `git rev-parse HEAD` when the result is written, not when the run starts, so the field dates the write and nothing else. This matters for the sibling comparisons above, which span at least three harness commits.

A single-rank run cannot falsify a claim about coordination. The most quotable sentence available here — that AgentMPI’s overhead is a fifth of a percent — is measured on the configuration that uses none of AgentMPI. It is the right baseline for exactly one question, whether the framework’s bookkeeping is a material tax on an agent workload, and the answer is no by two orders of magnitude. It is the wrong evidence for any question about whether the coordination mechanisms are worth their cost, and the four eight-rank cells are where that is decided.